

Using an ALB Behind Amazon CloudFront with a Custom HTTP Header to Control Access

Chris Samugana

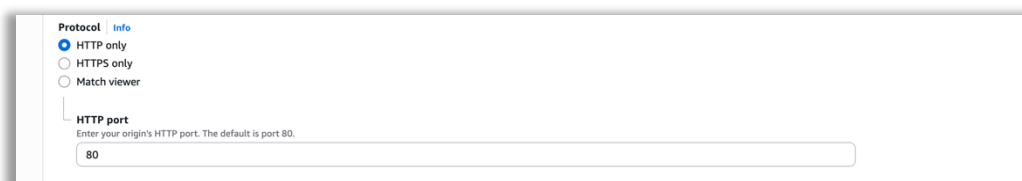
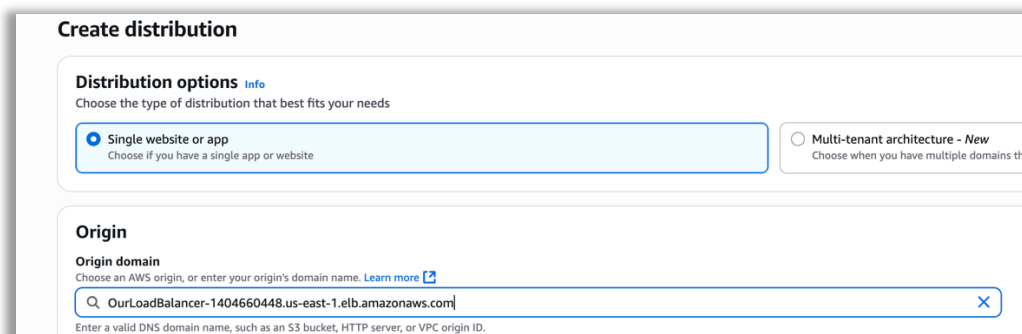
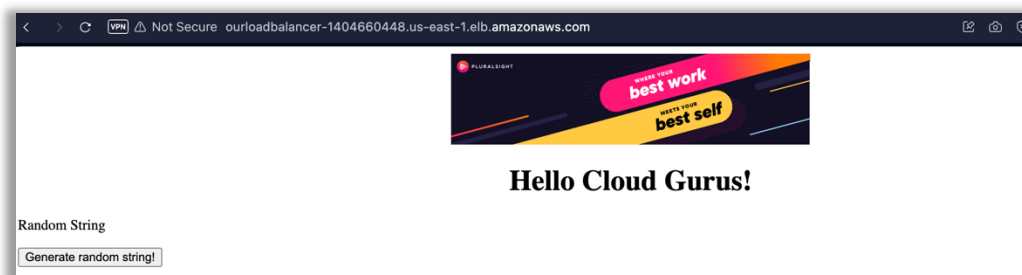
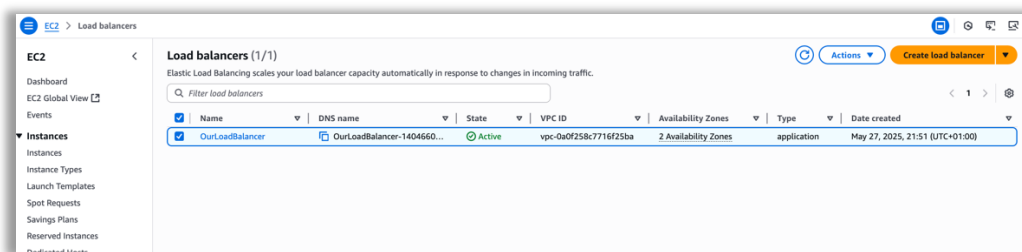
Introduction

In this project, we implemented a more secure web application architecture by integrating Amazon CloudFront, Application Load Balancer (ALB), and Amazon EC2. The focus was to restrict direct public access to the ALB by requiring a custom HTTP header, allowing only requests from CloudFront.

Solution Overview

To restrict public traffic and improve security posture:

- We fronted our ALB with a CloudFront distribution.
- Configured CloudFront to include a custom HTTP header in requests.
- Modified ALB listener rules to accept only requests with this header.
- All other requests received a 403 Forbidden response.



Step 1: Testing the Web Application

Before locking down access, we verified that the application was functioning correctly.

1. Navigated to the Amazon EC2 console.
2. Selected Load Balancers from the Load Balancing menu.
3. Identified the ALB named OurLoadBalancer.
4. Copied the ALB's DNS name and opened it in a browser using HTTP.
5. Verified the simple web application loaded and generated a random string when the button was clicked.

This screenshot shows the 'Add custom header' section of the Amazon CloudFront console. It includes a text input for the origin name, a table for custom headers with columns for 'Header name' and 'Value', and a section for 'Enable Origin Shield' with radio buttons for 'No' (selected) and 'Yes'. There is also an 'Add header' button and an 'Additional settings' link.

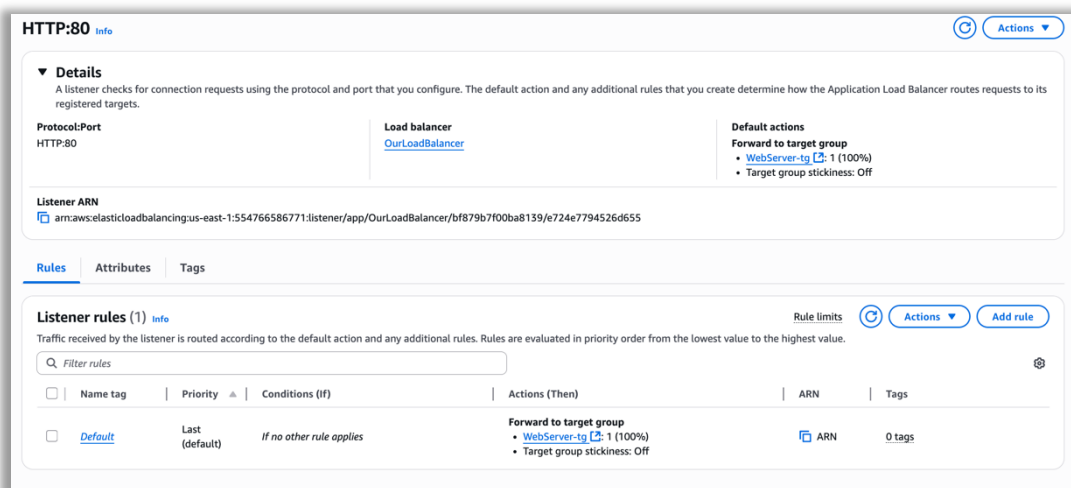
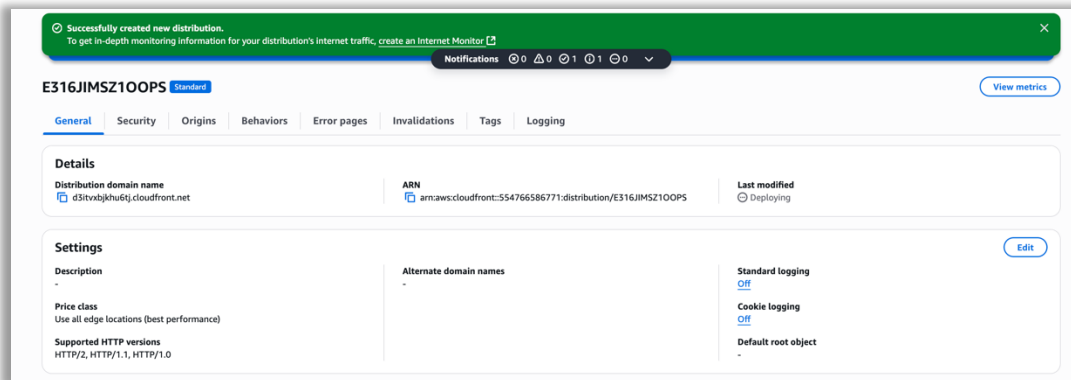
This screenshot shows the 'Viewer protocol policy' section of the Amazon CloudFront console. It includes radio buttons for 'HTTP and HTTPS', 'Redirect HTTP to HTTPS' (selected), and 'HTTPS only'. Below this are sections for 'Allowed HTTP methods', 'Cache HTTP methods', 'Allow gRPC requests over HTTP/2', 'Restrict viewer access', and 'Cache key and origin requests'. The 'Cache policy' dropdown is set to 'CachingOptimized'.

Step 2: Create the CloudFront Distribution

To securely deliver content and restrict access via a custom header:

1. Opened the Amazon CloudFront console.
2. Navigated to Distributions and selected Create a CloudFront distribution.
3. Under Origin settings:
 - For Origin domain, selected or manually entered the ALB's DNS name.
 - Set Protocol to HTTP only due to sandbox limitations.
4. Added a Custom HTTP Header:
 - Name: X-Custom-Secret
 - Value: a random string (e.g., abc123xyz). Saved this value for later use.
5. Under Default cache behavior:
 - Set Viewer protocol policy to Redirect HTTP to HTTPS.
 - Chose CachingOptimized for cache policy.

6. Skipped WAF configuration and left remaining settings default.
7. Clicked Create distribution and waited for it to deploy.



Step 3: Configure the ALB Listener Rule

To ensure only requests with the custom header could reach the backend:

1. Returned to the Amazon EC2 console > Load Balancers.
2. Selected the ALB and navigated to the Listeners tab.
3. Chose the HTTP:80 listener and clicked View/edit rules.
4. Added a new rule:
 - Name: Header Check
 - Condition:
 - HTTP header: X-Custom-Secret
 - Value: matched the one set in CloudFront
 - Action:
 - Forward to target group: WebServer-tg
 - Priority: 1
5. Saved the rule.
6. Edited the Default rule to:
 - Return fixed response with 403 Forbidden
 - Custom message: "Missing required HTTP Header"

Add rule Info

Define the rule and then review it in the context of the other rules on this listener.

▼ Listener details: HTTP:80

Protocol-Port
HTTP:80

Load balancer
[OurLoadBalancer](#)

Default actions
Forward to target group

- WebServer-tg [?](#): 1 (100%)
- Target group stickiness: Off

Listener ARN
[arn:aws:elasticloadbalancing:us-east-1:554766596771:listener/app/OurLoadBalancer/bf879b7f00ba8139/c724e77945264655](#)

▼ Name and tags Info

Tags can help you manage, identify, organize, search for and filter resources.

Name

[Add additional tags](#)

Add condition Rule limits

✕

Rule condition types
Route traffic based on the condition type of each request. Each rule can include one of each of the following conditions: host-header, path, http-request-method and source-ip. Each rule can include one or more of each of the following conditions: http-header and query-string.

HTTP header ▼

HTTP header name
Rule will assess requests containing this header to confirm matching values. Not case sensitive. Wildcards not supported.

✕

Valid characters are a-z, A-Z, 0-9 and [special characters](#). HTTP header name must be 1-40 characters. Character count: 15/40

HTTP header value
Enter strings to compare against the HTTP header value. Not case sensitive.

is

✕

Valid characters are a-z, A-Z, 0-9 and [special characters](#). HTTP header value must be 1-128 characters.

[Add new value](#)

You can add up to 4 more condition values for this rule.

Cancel

Confirm

Conditions (1) Info

[Rule limits](#) [Edit](#) [Delete](#) [Add condition](#)

Requests reaching this rule must match all specified conditions for the rule to apply. At least 1 condition is required.

HTTP header (1) Info

☐

If
HTTP header: X-Custom-Secret

is
12345

Actions Info

These actions will be applied to requests matching the rule conditions.

Action types

Routing actions

☒ Forward to target groups

☐ Redirect to URL

☐ Return fixed response

Forward to target group Info

Choose a target group and specify routing weight or [Create target group](#) [?](#)

Target group

Target type: Instance, IPv4

HTTP ▼

Weight

0-999

Percent
100%

[Add target group](#)

You can add up to 4 more target groups.

Target group stickiness Info

Enables the load balancer to bind a user's session to a specific target group. To use stickiness the client must support cookies. If you want to bind a user's session to a specific target, turn on the Target Group attribute Stickiness.

☐ Turn on target group stickiness

Set rule priority [Info](#)

Each rule has a priority. The default rule is evaluated last. You can change the priority of a non-default rule at any time. You can't change the priority of the default rule.

► Listener details: HTTP:80

Listener rules (2) [Info](#) Rule limits Reset priorities Add gap between priorities ▼

Traffic received by the listener is routed according to the default action and any additional rules. Rules are evaluated in priority order from the lowest value to the highest value.

Name tag	Priority	Conditions (If)	Actions (Then)	ARN	Tags
Header Check	1 Priority value must be 1-50,000.	HTTP Header X-Custom-Secret is 12345	Forward to target group - WebServer-tg: 1 (100%) - Target group stickiness: Off	Pending	1 tag
Default	Last (default)	If no other rule applies	Forward to target group - WebServer-tg: 1 (100%) - Target group stickiness: Off	ARN	0 tags

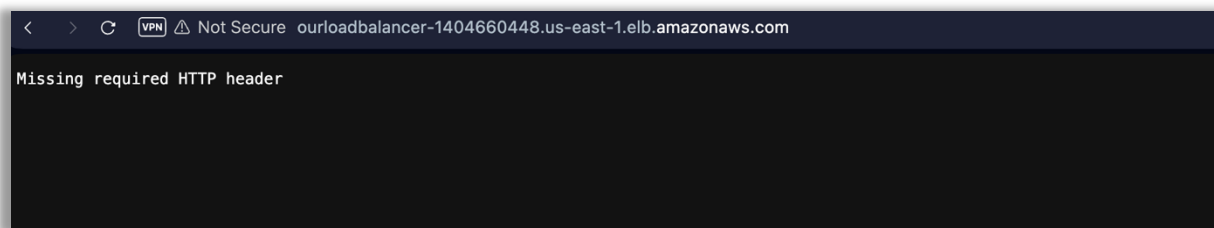
Cancel Previous Next

Default actions [Info](#)

The default action is used if no other rules apply. Choose the default action for traffic on this listener.

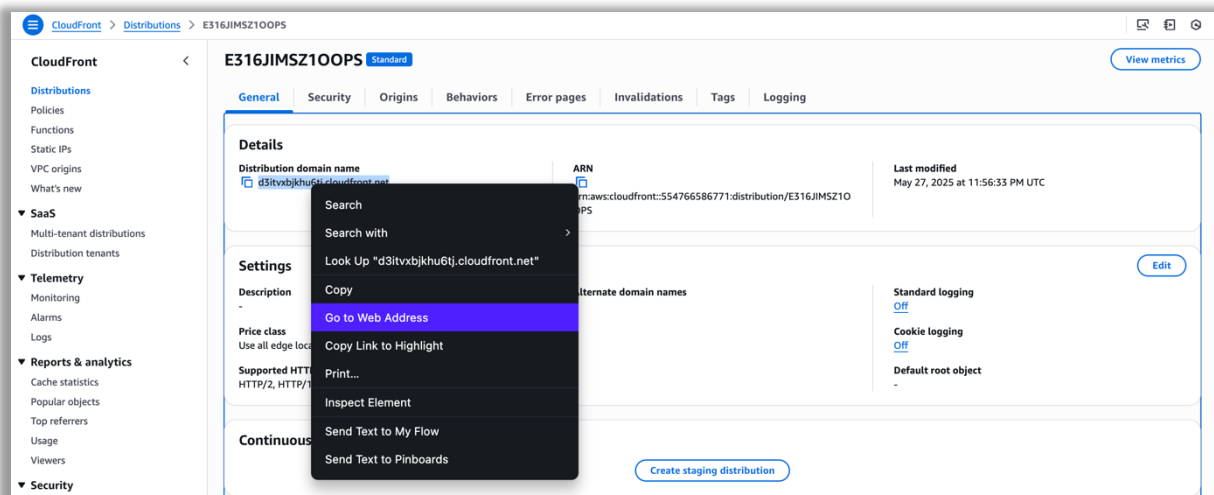
Routing actions

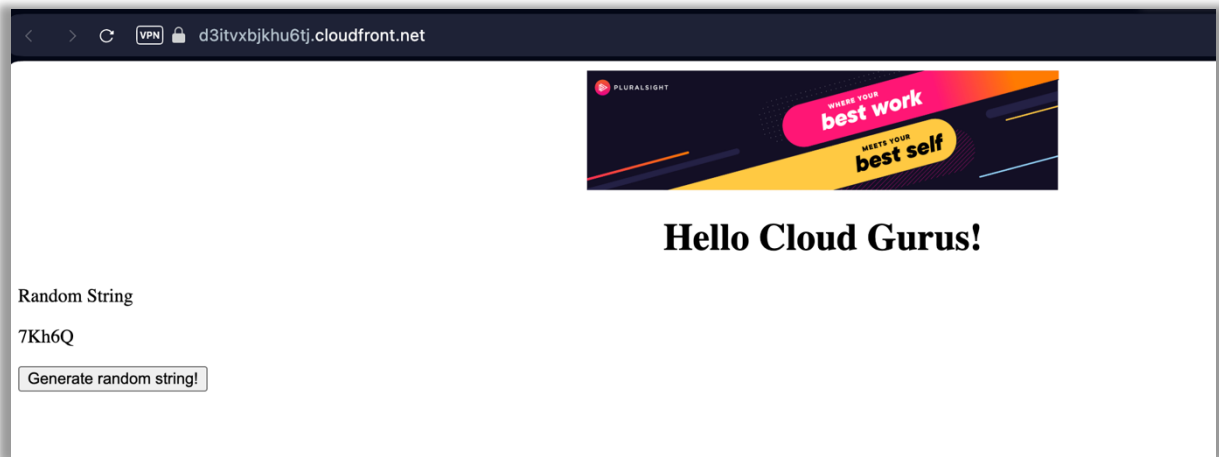
☐ Forward to target groups
☐ Redirect to URL
☒ Return fixed response



Step 4: Testing the Configuration

1. Navigated directly to the ALB's DNS in the browser:
 - Result: Received 403 Forbidden with custom message (as expected).
2. Navigated to the CloudFront distribution URL:
 - Example: <https://d8oknyay0fspy.cloudfront.net>
 - Result: Application loaded successfully and functioned normally.





Conclusion

- Deployed a simple web app behind an Application Load Balancer.
- Created a CloudFront distribution to serve as a secure front.
- Implemented a custom HTTP header to restrict direct ALB access.
- Tested and validated that the application is only accessible through CloudFront.